

Git Branching, Merging, and Reset Lab Report

Student: Majd Alharbi | Repository: WebBootcamp2

Lab Objective: Set up a Git repository, work with branches, merge and push changes, inspect file differences, and practice mixed and hard resets.

1. Working Directory

All commands were executed from:

C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P

Step 1 - Set up and publish the repository

I created the README file, initialized Git, created the first commit, renamed the default branch to main, connected the repository to GitHub, and pushed main.

```
echo "# WebBootcamp2" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/MajdAlharbi/WebBootcamp2.git
git push -u origin main
```

Result: The local repository was connected to GitHub, and main tracked origin/main.

```
PS C:\Users\majd9> cd C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> echo "# WebBootcamp2" >> README.md
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git init
Initialized empty Git repository in C:/Users/majd9/OneDrive/Desktop/WebBootcamp/5rd_P/.git/
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git add README.md
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git commit -m "first commit"
[master (root-commit) 729396d] first commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git branch -M main
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git remote add origin https://github.com/MajdAlharbi/WebBootcamp2.git
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 244 bytes | 244.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/MajdAlharbi/WebBootcamp2.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Step 2 - Verify the repository state

I checked that the working tree was clean and confirmed the active branch.

```
git status
git branch
```

Result: The repository was clean, and main was active.

```
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git branch
* main
```

Step 3 - Create practice branches

I used git switch -c from the class and git checkout -b from the lab instructions.

```
git switch -c "Majd"
git checkout -b login-feature
git branch
```

Result: The branches were main, Majd, and login-feature. The active branch was login-feature.

```
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git switch -c "Majd"
>> "
git: 'switch -cMajd'
' is not a git command. See 'git --help'.
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git switch -c "Majd"
Switched to a new branch 'Majd'
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git checkout -b login-feature
Switched to a new branch 'login-feature'
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git branch
  Majd
* login-feature
  main
```

Step 4 - Create and commit the login feature

Inside login-feature, I created login.py, added code, staged it, and committed the feature.

```
ni login.py
code login.py
git status
git diff
git add .
git commit -m "Add login feature"
```

Result: Commit b50160e added login.py with 10 lines. Because the file was initially untracked, git diff showed no content before staging.

```
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> ni login.py

Directory: C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P


Mode                LastWriteTime         Length Name
----                -
-a-----          8/2/2026   6:14 AM              0 login.py

PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> code login.py
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git status
On branch login-feature
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    login.py

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git diff
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> code login.py
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git diff
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git add .
```

Step 5 - Merge the feature into main

I switched to main and merged login-feature.

```
git switch main
git merge login-feature
git log --oneline
```

Result: Git completed a fast-forward merge, and login.py became part of main.

```
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git merge login-feature
Updating 729396d..b50160e
Fast-forward
 login.py | 10 ++++++++
 1 file changed, 10 insertions(+)
 create mode 100644 login.py
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git log --oneline
b50160e (HEAD -> main, login-feature) Add login feature
729396d (origin/main, Majd) first commit
```

Step 6 - Push the merged work and feature branch

I pushed the updated main branch and then published login-feature to GitHub.

```
git push origin main
git push -u origin login-feature
```

Result: Both main and login-feature were available on GitHub.

```
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 432 bytes | 432.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/MajdAlharbi/WebBootcamp2.git
 729396d..b50160e  main -> main
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git push -u origin login-feature
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'login-feature' on GitHub by visiting:
remote:   https://github.com/MajdAlharbi/WebBootcamp2/pull/new/login-feature
remote:
To https://github.com/MajdAlharbi/WebBootcamp2.git
 * [new branch]      login-feature -> login-feature
branch 'login-feature' set up to track 'origin/login-feature'.
```

Step 7 - Inspect a README change and practice a mixed reset

I edited README.md, inspected the change, committed it, and used git reset HEAD~1. This removed the latest commit but kept the README modification in the working directory.

```
notepad .\README.md
git diff
git add .
git commit -m "test git reset"
git log --oneline
git reset HEAD~1
git status
```

```
git log --oneline
```

Result: The test commit disappeared from the log, while README.md remained modified and unstaged. The change appeared as binary because Notepad used a different file encoding.

```
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> notepad .\README.md
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git diff
diff --git a/README.md b/README.md
index 09f9173..23e0475 100644
Binary files a/README.md and b/README.md differ
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git add .
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git commit -m "test git reset"
[main f22177b] test git reset
1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git log --oneline
f22177b (HEAD -> main) test git reset
b50160e (origin/main, origin/login-feature, login-feature) Add login feature
729396d (Majd) first commit
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git reset HEAD~1
Unstaged changes after reset:
M      README.md
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git log --oneline
b50160e (HEAD -> main, origin/main, origin/login-feature, login-feature) Add login feature
729396d (Majd) first commit
```

Step 8 - Practice a hard reset

I committed the README change again, then used `git reset --hard HEAD~1` to remove both the commit and its file changes.

```
notepad .\README.md
git add .
git commit -m "test git reset again"
git log
git reset --hard HEAD~1
git log
```

Result: HEAD returned to b50160e, and the test commit and its file changes were removed.

```

PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> notepad .\README.md
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git add .
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git commit -m "test git reset again"
[main b3ce840] test git reset again
1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git log
commit b3ce840b8c4b29107a67a378ecd994a37f6cdd (HEAD -> main)
Author: MAJD <majd9alharbi@gmail.com>
Date: Sun Aug 2 06:26:22 2026 +0300

    test git reset again

commit b50160ec0ea96a63eccc68b0857f97d6d80bcdec (origin/main, origin/login-feature, login-feature)
Author: MAJD <majd9alharbi@gmail.com>
Date: Sun Aug 2 06:16:55 2026 +0300

    Add login feature

commit 729396daae1d9ed3d145c17982789550ef7e4e3e (Majd)
Author: MAJD <majd9alharbi@gmail.com>
Date: Sun Aug 2 06:12:05 2026 +0300

    first commit
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git reset --hard HEAD~1
HEAD is now at b50160e Add login feature
PS C:\Users\majd9\OneDrive\Desktop\WebBootcamp\5rd_P> git log
commit b50160ec0ea96a63eccc68b0857f97d6d80bcdec (HEAD -> main, origin/main, origin/login-feature, login-feature)
Author: MAJD <majd9alharbi@gmail.com>
Date: Sun Aug 2 06:16:55 2026 +0300

    Add login feature

commit 729396daae1d9ed3d145c17982789550ef7e4e3e (Majd)
Author: MAJD <majd9alharbi@gmail.com>
Date: Sun Aug 2 06:12:05 2026 +0300

    first commit

```

2. Final Git State

Reference	Commit	Meaning
HEAD -> main	b50160e	Local main points to Add login feature.
origin/main	b50160e	GitHub main contains the login feature.
origin/login-feature	b50160e	The remote feature branch points to the same commit.
Majd	729396d	The local Majd branch still points to the first commit.

3. Key Takeaways

- **git add** - stages changes for the next commit.
- **git commit** - saves staged changes in local Git history.
- **git push** - uploads local commits or branches to GitHub.
- **git merge** - combines another branch into the current branch.
- **git diff** - shows exact changes in tracked files.
- **git reset HEAD~1** - removes the latest commit but keeps its file changes.
- **git reset --hard HEAD~1** - removes the latest commit and discards its file changes.

4. Conclusion

The lab was completed successfully. I created and published a repository, practiced branch creation and switching, merged a feature into main, pushed the work to GitHub, inspected changes, and tested mixed and hard reset behavior.